

Problema dello Zaino 0/1

Risoluzione con programmazione dinamica



Il problema

DEFINIZIONE

n oggetti con peso w_i e valore v_i , capacità W : scegliere il sottoinsieme di valore massimo che non superi la capacità. Ogni oggetto 0 o 1 volte.

$$\max \sum v_i x_i$$

$$\text{s.t. } \sum w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

ESEMPIO A 3 OGGETTI · $W = 10$

oggetto	peso	valore	densità
1	6	60	10
2	5	45	9
3	5	45	9

60

greedy per densità

90

ottimo

Il greedy per densità prende il primo e si ferma a 60. L'ottimo è 90.

Perché la PD si applica

$K[i,c]$ = valore ottimo con i primi i oggetti e capacità c.

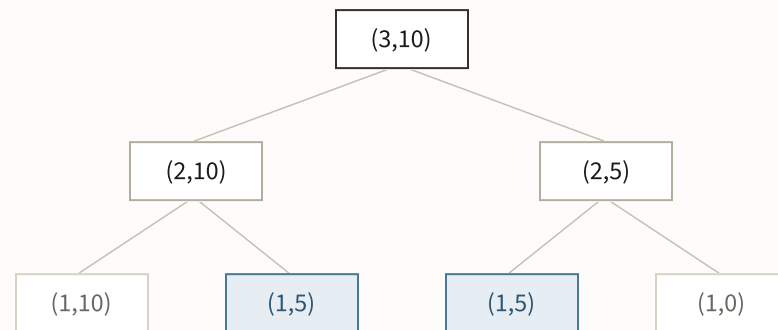
1 Sottostruttura ottima

Un ottimo di (i,c) contiene un ottimo di $(i-1,c)$ oppure di $(i-1,c-w_i)$. Argomento taglia-e-incolla.

2 Lo stesso sottoproblema più volte

$(1,5)$ si raggiunge sia prendendo l'oggetto 2 sia prendendo il 3, perché $w_2 = w_3 = 5$.

ALBERO DI RICORSIONE



i due $(1,5)$ sono lo stesso sottoproblema

I due algoritmi

RICORRENZA

$$K[i,c] = \begin{cases} 0 & \text{se } i = 0 \\ K[i-1,c] & \text{se } w_i > c \\ \max\{ K[i-1,c], v_i + K[i-1,c-w_i] \} & \text{se } w_i \leq c \end{cases}$$

KNAPSACK-VALUE

Riempie la tabella dal basso: l'ottimo è $K[n,W]$.

KNAPSACK-SOLUTION

La ripercorre all'indietro da (n,W) : l'oggetto i è stato preso se e solo se $K[i,c] \neq K[i-1,c]$. Non ricalcola nulla.

TABELLA K DELL'ESEMPIO · C DA 0 A 10

i	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	60	60	60	60	60
2	0	0	0	0	0	45	60	60	60	60	60
3	0	0	0	0	0	45	60	60	60	60	90

cammino a ritroso da $(3,10)$ · oggetti presi: 2 e 3

Analisi asintotica

$$\Theta(nW)$$

TEMPO

$$\Theta(nW)$$

SPAZIO

È un Θ e non un O : il corpo del ciclo più annidato gira esattamente $n(W+1)$ volte, non c'è caso migliore né peggiore.

$\Theta(nW)$ è lineare nel *valore* di W , non nella lunghezza della sua codifica: aggiungo un bit a W e il tempo raddoppia. È pseudo-polinomiale.

Implementazione e variante

IMPLEMENTAZIONE

Java 21, batch da/per file, celle a 64 bit contro l'overflow.

Per la riga i serve solo la riga $i-1$: se ne tiene una sola e la si aggiorna in luogo, percorrendo c da W verso il basso (il verso è essenziale: in senso crescente l'oggetto verrebbe preso più volte).

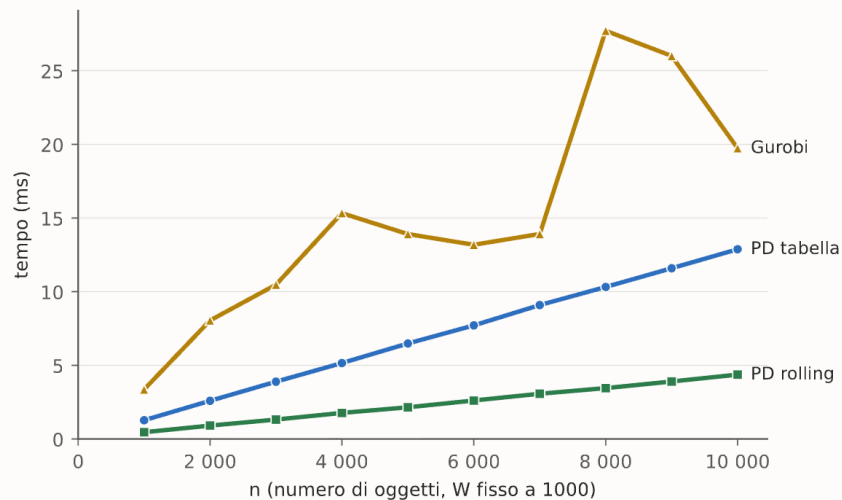
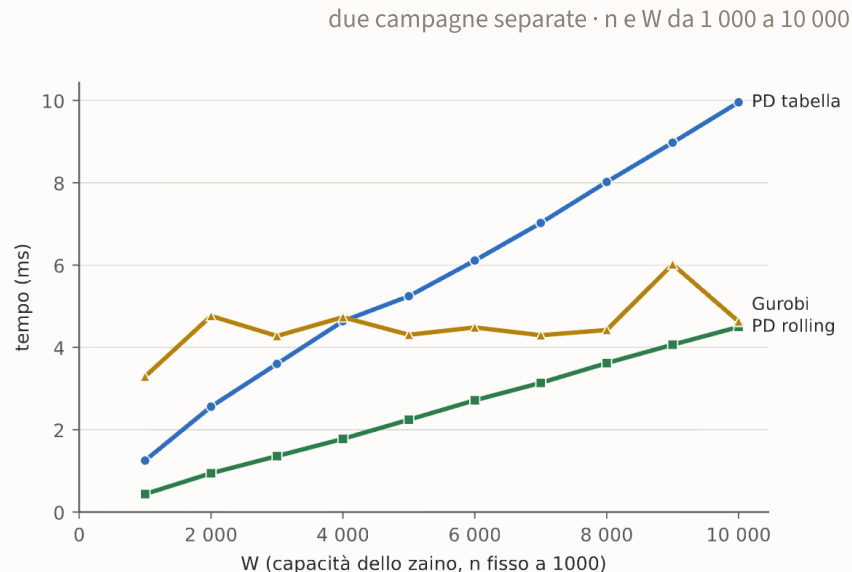
CICLO INTERNO DI KNAPSACK-ROLLING

```
for  $c \leftarrow W$  downto  $w[i]$ 
  if  $K[c-w[i]] + v[i] > K[c]$ 
    then  $K[c] \leftarrow K[c-w[i]] + v[i]$ 
```

LE DUE VARIANTI A CONFRONTO

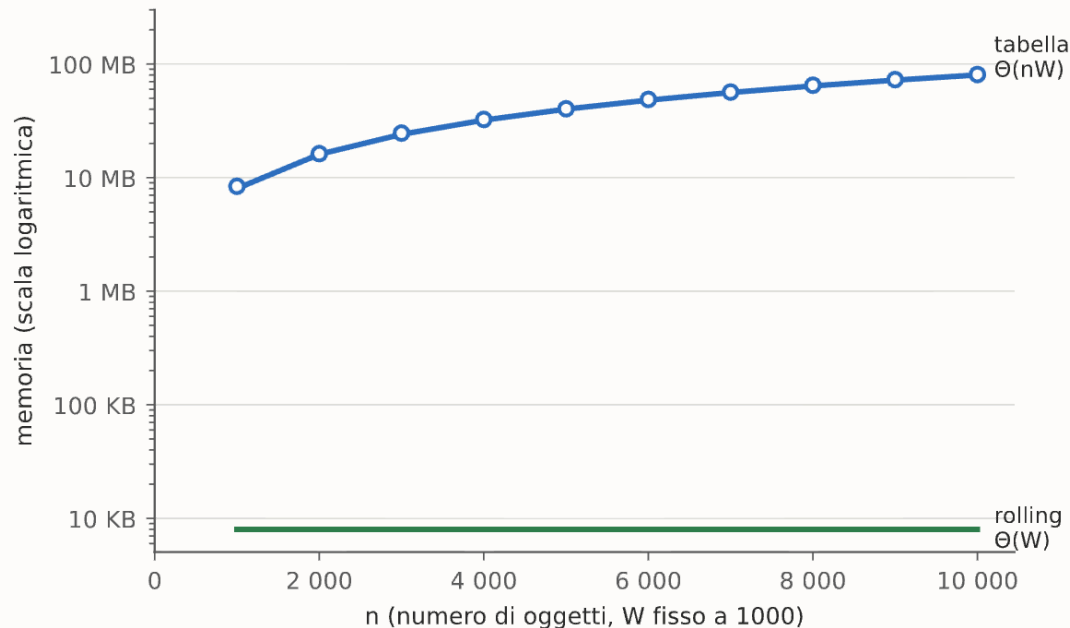
variante	spazio	ricostruzione
Tabella	$\Theta(nW)$	sì
Rolling	$\Theta(W)$	solo il valore

Sperimentazione — il tempo

tempo vs n · W fisso a 1000tempo vs W · n fisso a 1000

A capacità fissa il tempo cresce in proporzione a n ; al crescere di W la PD rallenta mentre il solver resta piatto — la pseudo-polinomialità vista dal vivo.

Sperimentazione — la memoria e il limite



memoria · scala logaritmica

La misura sull'heap segue la formula $(n+1)(W+1) \cdot 8$ byte; il rolling sta quattro ordini di grandezza più in basso.

$N = 2000 \cdot \text{HEAP } 2 \text{ GB}$

W	tabella	rolling
100 000	1,5 GB 0,81 s	781 KB 0,09 s
300 000	4,5 GB OOM	2,3 MB 0,28 s
1 000 000	14,9 GB OOM	7,6 MB 0,93 s

Empirico contro asintotico

0,9999

R^2 al crescere di n

0,9966

R^2 al crescere di W

0,43 ns

per cella · campagna n

0,45 ns

per cella · campagna W

R^2 della retta ai minimi quadrati sull'algoritmo base: il tempo cresce linearmente in ciascuno dei due fattori.

Tempo per cella della variante a spazio ridotto: lo stesso numero in due campagne indipendenti. La previsione è confermata anche nella costante, non solo nell'andamento.

Due risultati

1 L'analisi predice, e anche nella costante

Il tempo cresce linearmente in ciascuno dei due fattori (R^2 fino a 0,9999) e la memoria misurata sull'heap segue $(n+1)(W+1) \cdot 8$ byte. Non solo l'andamento: azzecca il valore.

2 È W a decidere, e cede prima la memoria

A stabilire quali istanze siano trattabili non è il numero di oggetti ma la capacità — e il limite arriva prima sulla memoria che sul tempo: a $W = 300\,000$ la tabella vorrebbe 4,5 GB e non parte, il rolling chiude in 2,3 MB e un quarto di secondo. La variante a spazio ridotto non è un'ottimizzazione: è ciò che rende risolvibili quelle istanze.

Correttezza: 305 test contro la forza bruta e un solver ILP come oracolo esterno, 20 punti su 20.

Segue la demo: tabella K ispezionabile, ricostruzione, confronto fra varianti.

Codice e riproduzione degli esperimenti

github.com/UniMarcoCesari/knapsack-dp

La campagna lascia dietro di sé i propri dati: per ogni punto delle curve un file CSV registra tempi, memoria, valore ottimo e il seme con cui l'istanza è stata generata. Da quei file si rigenerano le figure di questa presentazione, e con lo stesso seme la misura si ripete identica.

Le istruzioni sono nel README del repository: come compilare, come lanciare i programmi, come rifare la campagna e le figure.